# 13 — Conversation Memory & News-Agent Conversational QA (2026-11-09 → 2026-11-11)

**Thesis link.** §5.4 limitation "RAG retrieval quality … can miss context needed for complex questions" and the FYP1 UX gap where every free-text message was handled standalone: follow-ups like *"How about Kuching?"* or *"Tell me more"* had no access to what the user had just asked or been shown.

## 1. What was built

### 1.1 Conversation memory

- `src/core/models.py` — `ConversationMessage` (user_id, role, content, interaction_type `qa | latest | digest_summary`, created_at). One row per turn; old rows are pruned on every save (keep `CONVERSATION_MAX_TURNS`×2 rows and `CONVERSATION_MAX_AGE_HOURS`, also covered by the retention cleanup).
- `src/storage/migrate.py` — `migrate_create_conversation_messages_table()` (metadata-create, dialect-portable) registered in `init_db`.
- `src/core/conversation_service.py` — `save_conversation_turn()` / `get_recent_conversation_turns()`; both no-ops when `CONVERSATION_MEMORY_ENABLED=false`.
- Account deletion (`028757a`): conversation rows are deleted with the user — explicit child deletes first (SQLite has no alterable FKs), consistent with doc 11's pattern.

### 1.2 Follow-up understanding (deterministic heuristics, no LLM)

`is_followup_query` / `expand_followup_query` decide whether a short message continues the previous turn and rewrite it into a self-contained query:

- Follow-up markers (EN + Bahasa Malaysia, e.g. "how about", "macam mana pula"), word-count guards (≥7 words = standalone), topic/location swap detection.
- Topic extraction prefers category labels + a topic-keyword set, strips leading greetings, ignores location names as topics, tolerates punctuation, filters aux-verb junk from wh-questions, and recovers the topic from a previous *"I couldn't find <topic> news…"* refusal.
- `TOPIC_SYNONYMS` (EN + BM, e.g. `banjir → flood`, `kebakaran → fire`) normalize queries so BM follow-ups match English article content.

### 1.3 News-agent integration

- `answer_news_question_with_history()` injects recent turns into the answer prompt; the handler routes follow-ups conversationally and shows a persistent typing indicator on slow paths (`5f717aa`).
- **Topic-relevance gate** (`0f71c18`, refined in `90d931d`): when the expanded query has a clear topic but the semantic top-k misses it (including all synonym surface forms) and the location does not match, the bot answers from topic-and-location-matching pool articles instead of refusing — this removed the false negatives found in the first benchmark round.
- **Refusal UX** (`90d931d`): rotating refusal phrasing, suggestions backed by real 48-hour articles for the asked location, tappable suggestion buttons, and the hardcoded "■■ News agent" header dropped from all templates.

### 1.4 Per-conversation shown-article dedup (`84794bd`)

- `ConversationMessage.shown_article_links` (JSON) + idempotent migration (SQLite + Postgres), registered in `init_db`.
- Assistant turns persist the article links actually displayed; later turns in the same conversation exclude them from retrieval (`exclude_article_links`).
- Graceful by design: if exclusion would leave nothing, the full set is kept, and the topic-relevance gate reverts exclusion when the on-topic article is one already shown — so *"tell me more about that story"* still works. Deliberate trade-off: re-asking details about a shown article is possible; re-asking the same broad question surfaces fresh stories instead.

## 2. Code map

| File | Role |
|---|---|
| `src/core/conversation_service.py` | turn save/load, prune, follow-up heuristics, topic/location extraction |
| `src/core/services.py` | `get_news_agent_response_for_user(..., exclude_article_links)` → `(html, shown_links)`; topic gate; refusal/fallback formatters |
| `src/bot/handlers.py` | history load, follow-up routing, dedup wiring, suggestion-button callback, typing indicator |
| `src/ai/summarizer.py` | `answer_news_question_with_history` prompt |
| `src/core/models.py`, `src/storage/migrate.py`, `src/storage/database.py` | model, migration, startup registration |
| `scripts/benchmark_conversation_qa.py` | tier 1 + tier 2 benchmark harness |

Config: `CONVERSATION_MEMORY_ENABLED` (default true), `CONVERSATION_MAX_TURNS` (3), `CONVERSATION_MAX_AGE_HOURS` (24).

## 3. Evaluation (2026-11-11, evidence under `benchmark/`)

Harness: `scripts/benchmark_conversation_qa.py`.

- **Tier 1 — deterministic heuristic tests** (no DB/LLM): 12/12 pass (`conversation_tier1_20261111T023338Z.json`) — location swap, topic swap, vague "tell me more", wh-questions, greeting prefix, punctuation, refusal recovery, BM follow-up, and standalone-query guards.
- **Tier 2 — end-to-end conversations** (production Supabase DB, read-only + LLM; manual 1–5 rubric along thesis dimensions, plus auto-metrics `no_repeat` novelty and abstention):
- Before the false-negative fix (…`012628Z`): relevance 2.31, coherence 4.08, factual 3.31, 3 false-negative turns, tier 1 9/10 (+2 documented limitations).
- After (…`023339Z` + `conversation_scores_20261111T023339Z.json`): relevance **3.62**, coherence **4.38**, factual **4.0**, false negatives **0**, tier 1 **12/12**, mean novelty 0.86.
- Novelty dips are follow-up-detail turns re-showing the referenced article — the deliberate trade-off in §1.4. The abstention auto-metric undercounts after the refusal templates stopped containing "couldn't find".

**Documented limitations.** Single AI rater for the rubric scores; remaining weak turns are LLM answer quality (unsupported figures in a temporal thread), a prompt/grounding issue rather than retrieval.

## 4. Reproduce

```
python -m compileall src
python scripts/benchmark_conversation_qa.py # tier 1 only
python scripts/benchmark_conversation_qa.py --tier2 # + end-to-end (needs warmed DB;
# set DATABASE_URL to Supabase for
# production-backed runs; read-only)
```

Live smoke: ask *"Any flood news in Sibu?"*, then *"Macam mana pula di Miri?"* — the second answer should carry the flood topic into Miri without repeating the Sibu articles.